

박재남

제가 풀어 온 문제와 그때 내린 선택을 사례 단위로 정리한 문서입니다. 지금까지 맡았던 모든 일을 빠짐없이 나열하기보다, 지금의 일하는 방식에 가장 큰 영향을 준 문제와 시간이 지나도 선명하게 남아 있는 장면을 골라 담았습니다.

최근 경력일수록 사례 단위로 자세히 적었고, 초기 경력은 흐름과 그 시기에 얻은 것 중심으로 짧게 남겼습니다.

생년월일

92.12.16

이메일

jnpark1623@gmail.com

전화

010-5550-1623

지역

Seoul, KR

LINKEDIN

<https://www.linkedin.com/in/jnpark1623>

DeepSearch

Backend Software Engineer (2023/12-현재) · Team Lead - Finance AI (2025/05-현재)

DeepSearch AI와 금융 데이터 엔진 Finorma에서 백엔드 엔지니어, PO, 팀 리드 역할을 함께 수행하고 있습니다. 외부 금융 데이터를 받아 오는 가장 아래 계층부터 그 데이터로 만드는 AI 제품과 조직의 우선순위까지가 제가 맡고 있는 범위입니다.

성격이 제각각인 외부 금융 데이터를 하나의 전달 체계로

수신 방식도, 고객이 원하는 전달 방식도 소스마다 달랐습니다.

Koscom·KRX 시세는 TCP로 실시간 수신해야 했고, NICE기업정보 재무데이터는 FTP로, 해외 브로커·데이터 소스 (Massive, Finnhub, Alpha Vantage, Alpaca 등)는 API로 들어왔으며, 일부 데이터는 스크래핑으로 모아야 했습니다. 받는 쪽 사정이 이렇게 다른데, 내보내는 쪽 요구도 콜백·TCP·API·SFTP로 갈렸습니다.

벤더 연동부터 수신 이후의 정형화·가공·적재, 재전달까지를 직접 맡았습니다. 소스별 차이는 수신 단에서 흡수하고, 내부 기준으로 한 번 정리한 데이터를 네 가지 형태로 다시 내보내는 구조를 만들어 소스가 늘어도 전달 계층은 그대로 유지되도록 했습니다.

금융기관·공공금융 조직 대상 제품이 공통으로 올라탈 수 있는 금융 데이터 파이프라인 기반이 되었습니다.

Koscom·KRX

NICE FTP

TCP

API

SFTP

Python

15분 지연을 걷어내고, 종가를 흔든 ETF 데이터 장애를 복구

실시간이라는 이름과 달리 시세가 약 15분 뒤에 도착하고 있었습니다.

레거시 실시간 시세 전달 구조의 지연은 그 데이터를 받아 쓰는 모든 고객 서비스의 품질에 그대로 얹혔습니다. 전달 경로의 병목을 찾아 개선해 지연을 초저지연에 가까운 수준까지 낮췄습니다.

가장 강하게 기억에 남은 장애는 ETF 거래량이 int 범위를 초과하면서 데이터가 누락된 건이었습니다. 종가에 영향을 주는 데이터였기 때문에 복구와 재발 방지를 동시에 가져가야 했습니다. 복제 테이블을 만들어 스키마를 float으로 재정의하고, Kafka 컬렉터를 제어한 상태에서 테이블을 스왑해 누락분을 되살렸습니다.

여기서 멈추지 않고 같은 상황을 한 번의 실행으로 복구할 수 있는 체계를 만들었으며, 데이터 가시성과 모니터링, 데이터 기준까지 함께 정리했습니다. 장애는 다시 나더라도 대응 시간이 짧아지도록 남겨 두는 편이 낫다고 판단했습니다.

시세 지연은 초저지연 수준으로 내려갔고, 데이터 장애는 원클릭 복구 체계와 모니터링 기준이 남는 형태로 마무리했습니다.

Kafka

schema migration

table swap

monitoring

흩어진 데이터 도메인 재정렬과 파이프라인 현대화

도메인과 엔지니어링 영역이 흩어져 있어, 무엇부터 손대야 하는지가 합의되지 않은 상태였습니다.

데이터 도메인의 경계를 다시 그리고, 레거시 파이프라인을 신규 파이프라인으로 옮기는 일을 맡았습니다. EKS 환경에서 Python과 Celery로 파이프라인을 운영하면서, 성격이 다른 데이터에는 다른 수집 방식을 적용했습니다.

Koscom·해외 시세처럼 흐름이 끊기면 안 되는 데이터는 Socket-to-Kafka로, 뉴스·공시 같은 비정형 실시간 데이터는 stateful sourcing으로 수집하도록 신규 파이프라인을 구축했습니다. 운영 중인 서비스를 멈추지 않고 옮기는 것이 전제였습니다.

재정렬한 데이터 도메인과 신규 파이프라인으로 현재까지 안정적으로 서비스하고 있습니다.

EKS

Python

Celery

Kafka

stateful sourcing

기존 클라이언트를 깨지 않는 API 전환과 Dagster·Medallion 데이터 계층

신규 아키텍처로 가면서도, 이미 붙어 있는 클라이언트는 그대로 동작해야 했습니다.

클라이언트에게 언제까지 맞춰 달라고 요구하는 대신, 기존 API 패턴이 redirect 방식으로 계속 동작하도록 설계한 뒤 신규 API로 이관했습니다. 전환 일정의 부담을 우리 쪽에서 흡수하는 선택이었습니다.

데이터 계층은 Dagster pipeline 위에 Medallion Architecture로 재구현해 계층을 나눠 운영하고 있습니다.

기존 클라이언트 호환성과 신규 아키텍처 전환을 동시에 관리하며 현재 구조를 운영하고 있습니다.

Dagster

Medallion Architecture

API redirect

은행권 협업과 금융 데이터의 AI 제품화

은행이 원한 것은 데이터 자체가 아니라 그 데이터로 만들 수 있는 서비스였습니다.

KakaoBank AI 투자 챗봇 협업에 깊게 관여했고, Shinhan Bank에는 DeepSearch 데이터를 공급했습니다. K Bank 투자 서비스 개편과 Hana Bank OneQ Pro MTS에서는 데이터를 어떻게 활용할지에 대한 기획 협업을 지원했습니다. 직접 구현한 범위와 협업·데이터 공급으로 참여한 범위는 구분해서 적었습니다.

같은 시기에 데이터 자산 자체를 제품으로 만드는 일도 함께 했습니다. 증권 시세·기업 재무 같은 정형 데이터, 뉴스·공시 같은 비정형 데이터, 유튜브·소셜처럼 채널이 특정되지 않는 데이터까지 모아 토픽을 추출하고 AI Ready 형태로 전처리·적재했습니다.

그 위에서 AI 분류와 생성형 콘텐츠 제작까지 구현했고, native LLM 호출부터 DAG, agentic loop까지 문제에 맞는 방식을 골라 제품화 단계로 연결했습니다.

금융 데이터 기반을 은행권 서비스 협업과 DeepSearch AI·Finorma의 AI 제품 실행으로 연결했습니다.

LLM

agentic loop

DAG

AI Ready 전처리

직군이 다른 사람들이 같은 우선순위를 보게 만드는 일

영업, 쿼트, 엔지니어링, 디자인, AI 서비스 기획이 한 조직 안에 있었습니다.

약 10~12인 규모 풀스택 조직의 리드와 PO를 맡으면서 목표와 우선순위, 딜리버리 구조를 정리하고 엔지니어링의 기술 방향까지 함께 책임졌습니다. 직군마다 성과를 재는 기준이 달랐기 때문에, 같은 문장으로 목표를 말할 수 있게 만드는 데 가장 많은 시간을 썼습니다.

조직 안에 남아 있던 비효율은 AX(AI Transformation)로 전환해, 구성원이 핵심 업무에 더 몰입할 수 있는 환경을 만들었습니다.

신규 ARR 약 7배 성장에 기여했고, 담당 팀 영역은 현재 DeepSearch 매출의 약 90%를 차지하는 수준으로 성장했습니다.

Miso, Inc.

Engineering Manager · 2023/11 — 2023/12

Engineering Manager로 합류했습니다. 회사는 Product 조직 형태였고 신규 사업 런칭 과제가 주어졌지만, 매트릭스 형태의 엔지니어링 조직은 기존 서비스 마이그레이션이 선행되어야 한다는 입장이었습니다. 두 조직의 이해와 우선순위가 좁혀지지 않는 상태가 이어져 비교적 빠르게 퇴직 의사를 밝히고 나왔습니다.

짧은 기간이었지만, 조직 구조와 우선순위 합의가 제품 실행 속도를 어디까지 좌우하는지 분명하게 확인한 시기였습니다.

Goodoc

Backend Engineer (O2O Squad) → PO 겸 Engineer (Goodoc Labs) · 2022/05 — 2023/11

O2O 스쿼드의 Backend Engineer로 약 600만 회원 규모의 병원 예약·접수 서비스를 개발·운영한 뒤, Goodoc Labs로 옮겨 PO 겸 Engineer로 제품 방향과 기술 실행을 함께 맡았습니다.

600만 회원 서비스의 실시간 접수 서버 구축

병원 예약·접수는 상태가 실시간으로 바뀌는 서비스였습니다.

Node.js·NestJS·TypeORM으로 서비스 백엔드를 개발하고 앱 서비스 API는 GraphQL로 제공했습니다. 여기에 실시간 접수 처리를 위한 Socket 기반 O2O 접수 서버를 신규로 구축해 병원과 사용자 양쪽의 접수 흐름을 이어 붙였고, EKS 환경에서 운영했습니다.

상대적으로 stateful하고 고가용성이 요구되는 서버였기 때문에, 초기 구축부터 운영까지의 판단을 직접 가져가야 했던 경험이었습니다.

약 600만 회원 규모 병원 예약·접수 서비스와 B2B2C 접수 흐름을 실시간으로 지원했습니다.

Node.js

NestJS

TypeORM

GraphQL

Socket

EKS

라이브 서비스를 멈추지 않은 v3 → v4 전환

이미 돌고 있는 서비스를 세우지 않고 아키텍처를 바꿔야 했습니다.

클라우드 기반 라이브 서비스 환경에서 대규모 버전 전환 과제를 맡아, 기존 레거시 환경에서 신규 아키텍처로 무중단 마이그레이션을 수행했습니다.

서비스 중단 없이 신규 버전과 아키텍처로 이관했습니다.

Node.js

NestJS

zero-downtime migration

인수인계 없이 넘겨받은 장애들

합류 직후, 매주 반복되는 접속 서버 장애를 배경 설명 없이 넘겨받았습니다.

메트릭도 인수인계도 충분하지 않은 상태에서 장애를 직접 추적해 DB CPU 병목과 비효율 쿼리를 원인으로 특정했습니다. 트래픽 패턴을 확인해 부하가 낮아지는 시간대를 골라 인덱스를 적용하는 방식으로, 서비스 영향을 최소화하면서 구조를 바꿨습니다.

비슷한 시기에 접근 정보조차 거의 남아 있지 않던 레거시 인증 EC2 장애도 있었습니다. 그 서버가 무슨 역할을 하는지 파악하는 것부터 시작해 AMI 복제와 패키지 버전 수정으로 우회 복구 경로를 만들어 서비스를 되살렸습니다.

매주 반복되던 접속 서버 장애를 안정화했고, 레거시 인증 서비스의 연속성도 회복했습니다.

DB tuning

index

EC2

AMI

Goodoc Labs에서 PO 겸 Engineer로

성장 조직에서는 무엇을 만들지 정하는 일까지 함께 해야 했습니다.

2023년 8월부터 11월까지 Goodoc Labs 성장 조직으로 이동해 PO 겸 Engineer로 일했습니다. 제품 방향 설정과 우선순위 조율을 백엔드 실행과 함께 가져가며 성장 조직의 제품·기술 의사결정에 참여했습니다.

엔지니어링 실행 범위를 제품 방향과 성장 문제 해결까지 넓힌 계기가 되었습니다.

Pentasecurity Systems

Lead Software Engineer / DX Team Lead · 2015/10 — 2022/05

데이터 암호화 플랫폼의 운영·관리 솔루션을 개발했고, 이후 4인 조직을 이끄는 DX Team Lead로 팀의 방향과 실행을 함께 책임졌습니다.

보안 제품은 한 번의 판단 실수가 곧바로 신뢰 문제로 이어지는 영역입니다. 6년 넘게 이 환경에서 일하며 신중한 의사결정과 부서 간 협업, 신뢰를 전제로 한 실행 방식을 몸에 익혔고, 2019년에는 사내 Pentastic Award를 받았습니다.

Media Solutions

Software Engineer (industrial service alternative) · 2012/10 — 2015/08

산업기능요원으로 근무하며 디지털 사이니지와 키오스크용 미디어 콘텐츠를 개발했습니다. 약 3년간 실제 현장에서 돌아가는 소프트웨어를 만들며 서비스형 소프트웨어 개발의 기본기와 현장 대응 경험을 쌓았습니다.

이력서 <https://www.mnpark.info/r/19a68f8ba8ce75123f267f00>